

Combinational Logic

Computer Architecture (ME-5319)

Phung Thanh Huy

Faculty of Mechanical Engineering
Ho Chi Minh City University of Technology (HCMUT) - VNUHCM

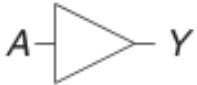
2026-v1



Logic Gates

Logic Gates and Truth Tables

BUF



$$Y = A$$

A	Y
0	0
1	1

AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

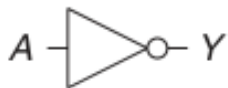
XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR



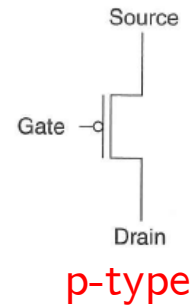
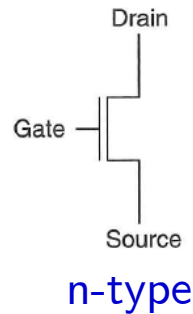
$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

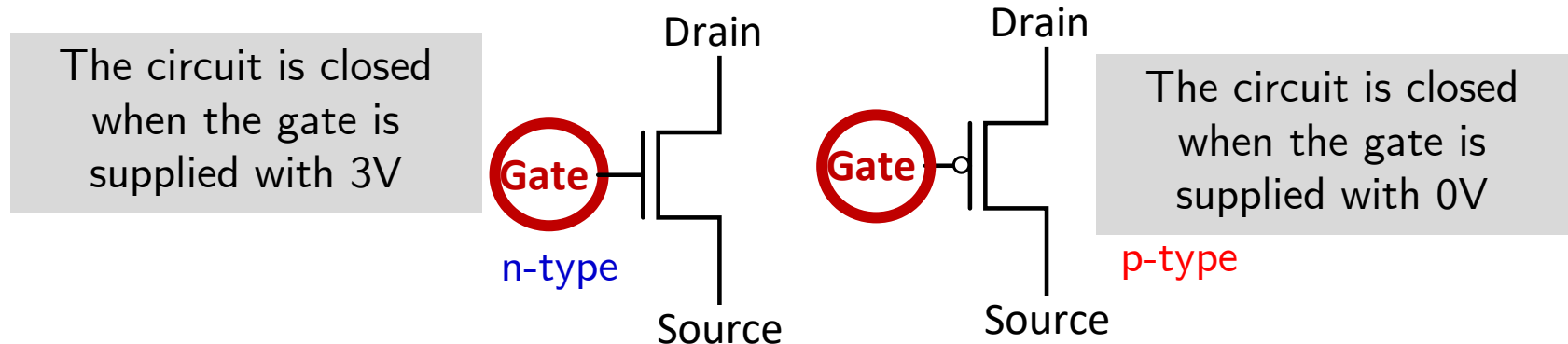
Logic Gates

Logic Gates and Truth Tables

- There are two types of MOS transistors: **n-type** and **p-type**



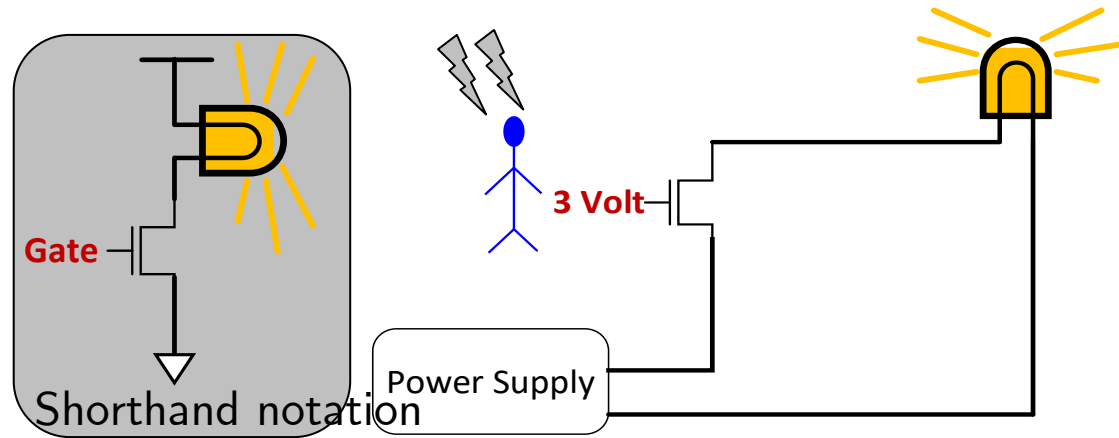
- They both operate “logically,” very similar to the way wall switches work



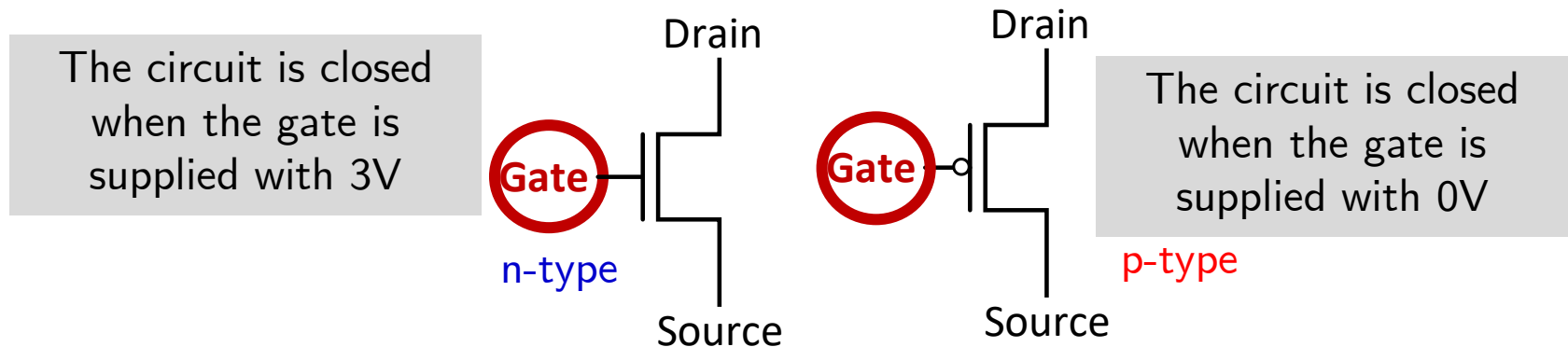
Logic Gates

Logic Gates and Truth Tables

- The **n-type** transistor in a circuit with a battery and a bulb



- The **p-type** transistor works in exactly the opposite fashion from the **n-type** transistor



Logic Gates

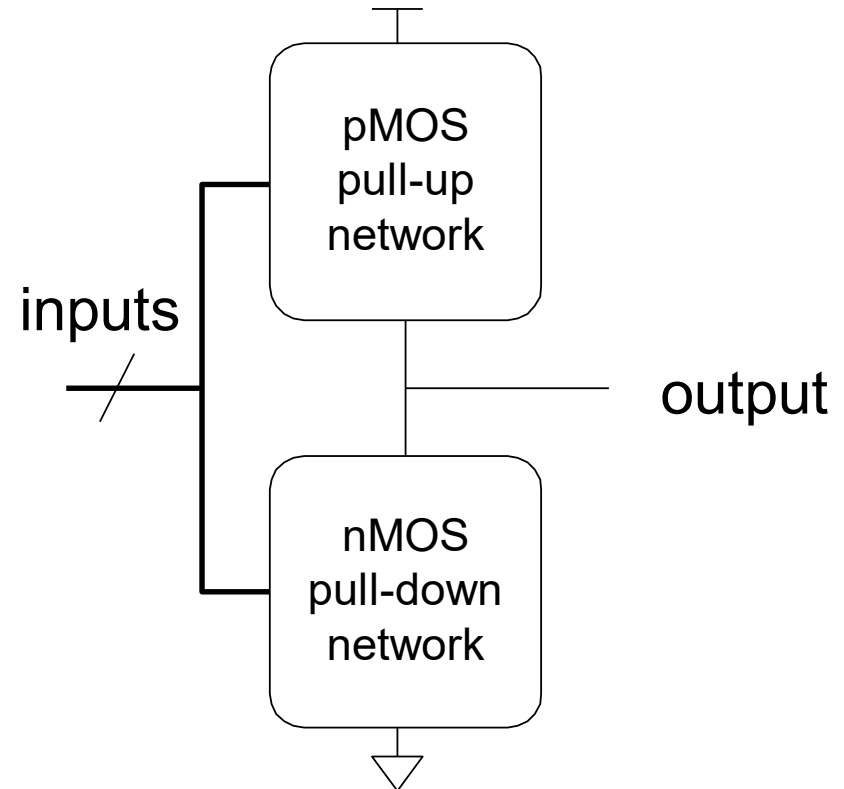
Logic Gates and Truth Tables

- The general form used to construct any inverting logic gate, such as: NOT, NAND, or NOR

- The networks may consist of transistors in series or in parallel
- When transistors are in **parallel**, the network is **ON** if **one** of the transistors is **ON**
- When transistors are in **series**, the network is **ON** only if **all** transistors are **ON**

pMOS transistors are used for pull-up

nMOS transistors are used for pull-down

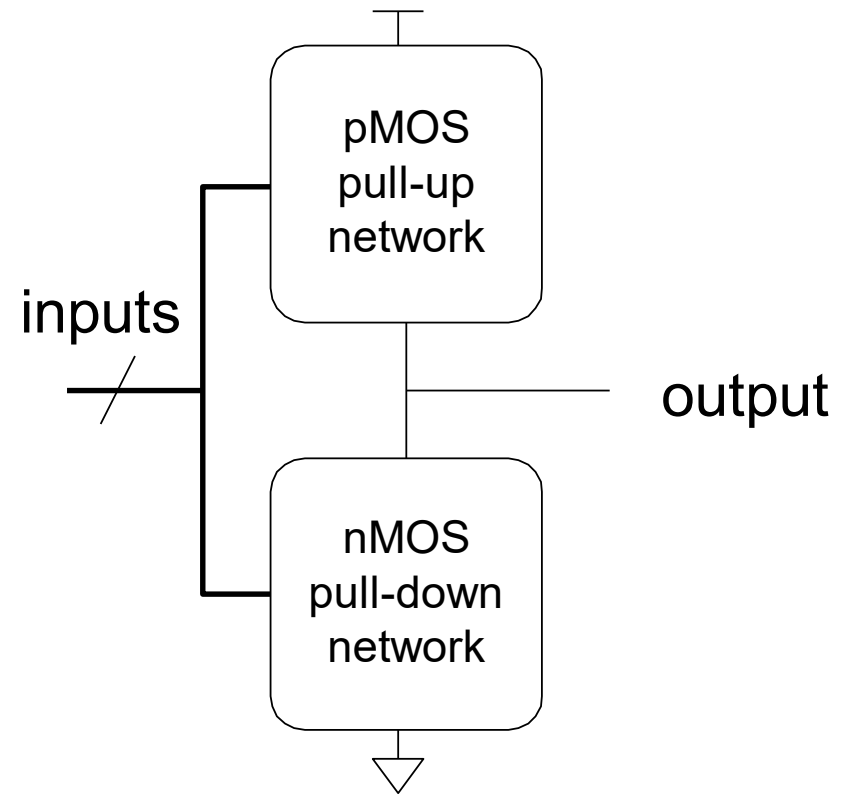


Logic Gates

Logic Gates and Truth Tables

- Exactly one network should be ON, and the other network should be OFF at any given time

- If both networks are ON at the same time, there is a **short circuit** → likely incorrect operation
- If both networks are OFF at the same time, the output is **floating** → undefined



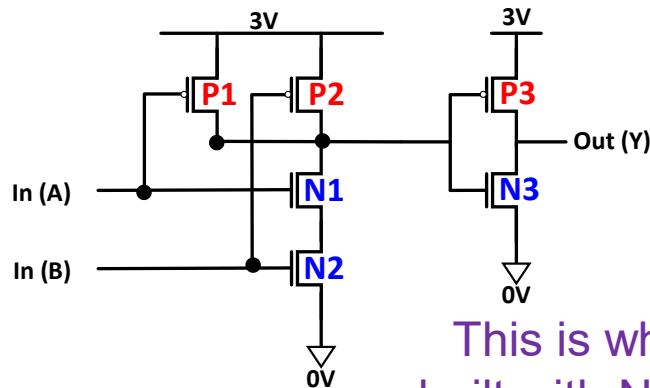
pMOS transistors are used for pull-up

nMOS transistors are used for pull-down

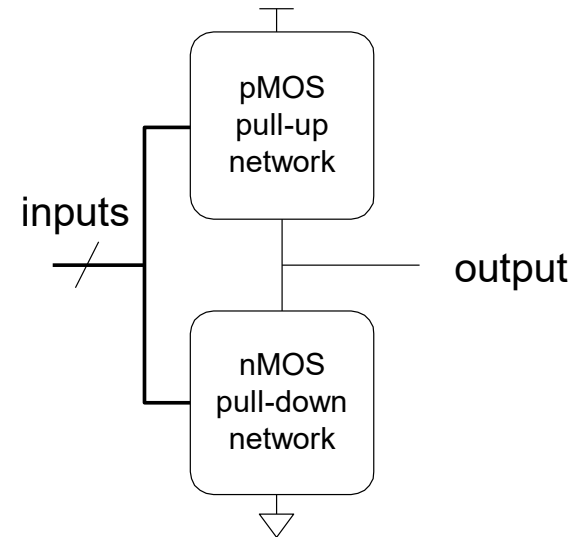
Logic Gates

Logic Gates and Truth Tables

- MOS transistors are **imperfect** switches
- pMOS transistors pass 1's well but 0's poorly (holes carry charge)
- nMOS transistors pass 0's well but 1's poorly (electrons carry charge)
- **p**MOS transistors are good at “pulling up” the output
- **n**MOS transistors are good at “pulling down” the output



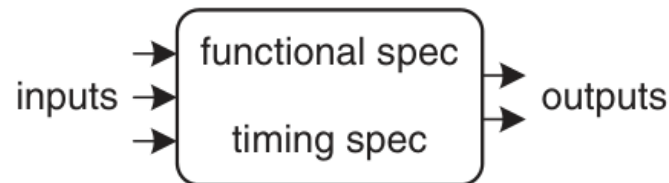
This is why AND is
built with NAND + NOT



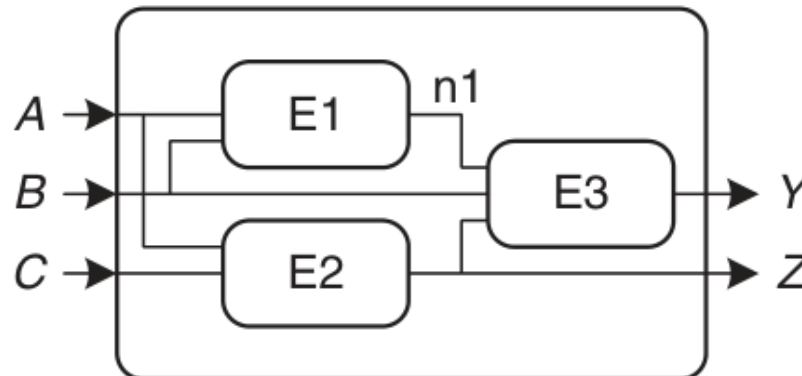
Introduction

Digital Circuits

- A circuit is a network that processes discrete-valued variables:
 - One or more discrete-valued input terminals.
 - One or more discrete-valued output terminals
 - A functional specification describing the relationship between inputs and outputs.
 - A timing specification describing the delay between inputs changing and outputs responding



- Inside a circuit:
 - Element: is itself a circuit with inputs and outputs, specification.
 - Node: Wire, whose voltage conveys a discrete-valued variable.

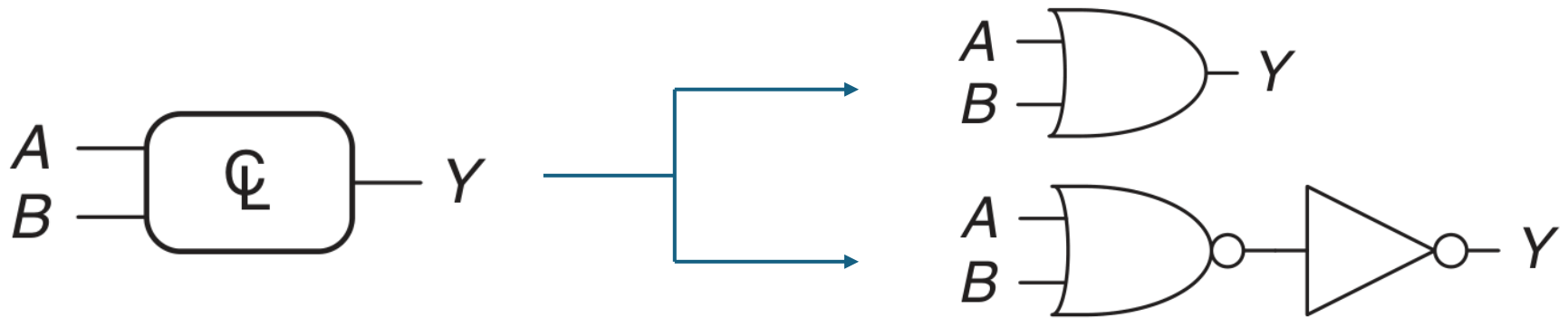


Introduction

Digital Circuits

- A digital circuit can be:
 - Combinational logic circuit: Memoryless (without information of the previous states).
 - Sequential logic circuit: with Memory (with information of the previous states).
- Example: A combinatory circuit

$$Y = F(A, B) = A + B$$

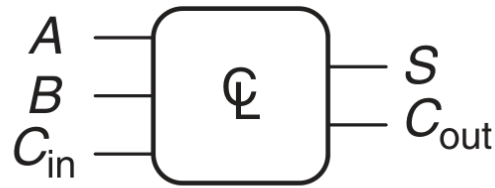


- A function can have different implementation forms

Introduction

Digital Circuits

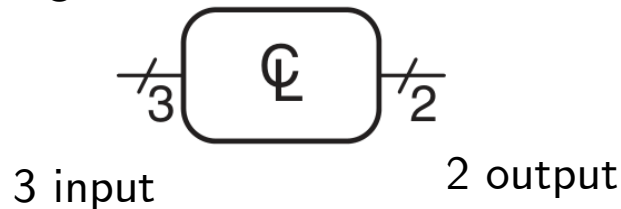
- Example: Full Adder circuit (Cộng đầy đủ - Cộng với nhớ)



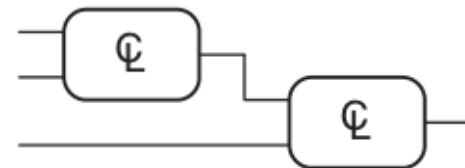
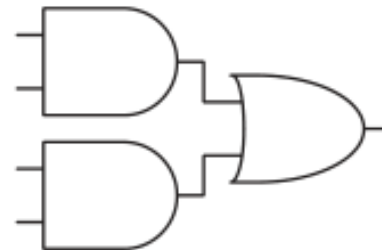
2 output 3 input
 $S = F(A, B, C_{in})$
 $C_{out} = G(A, B, C_{in})$

$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

- Bus: Bundle of multiple signals



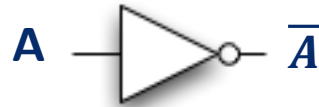
- Combinational Composition: build a large combinational circuit from the smaller ones.



Boolean Algebra

Simple Equations

$\bar{A}$ (reads “not A”) is 1 if A is 0



Complement of A:

A: true form

$\bar{A}$: complement form.

A	$\bar{A}$
0	1
1	0

$A \bullet B$ (reads “A and B”) is 1 if A and B are both 1



A, B, $\bar{A}$, $\bar{B}$, ...: literal

Product

Implicant: Product of literals

A	B	$A \bullet B$
0	0	0
0	1	0
1	0	0
1	1	1

$A + B$ (reads “A or B”) is 1 if either A or B is 1



Sum

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Boolean Algebra

Boolean Equations

- Complex equations are built from the 3 basic equations
- The order of operations (basic equations):
 - NOT has the highest precedence.
 - AND is the second, then OR
- A sample of Boolean equation:

$$F(A, B, C) = \bar{A}B + BC\bar{D}$$

- Minterms (m_i):
 - A product involving all the inputs.
 - Example, for $F(A, B, C)$:
 - $ABC, \bar{A}BC, A\bar{B}C, \dots$ are minterms
 - $AB, AC, B\bar{C}$ are not minterms.
- Maxterms (M_i):
 - A sum involving all the inputs.
 - Example, for $F(A, B, C)$:
 - $A + B + C, \bar{A} + B + C, A + \bar{B} + C, \dots$ are maxterms
 - $AB, AC, B\bar{C}$ are not minterms.

Boolean Algebra

Boolean Equations

- i in m_i and M_i : the number converted from the inputs.
- Example: $F(A, B, C)$
 - Let $ABC \equiv (b_3b_2b_1)_2$

A	B	C	i
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

Boolean Algebra

Sum-of-Products form (SOP) and Product-of-Sums form (POS)

- A function can be represented by product of minterms or sum of maxterms
- Example:

$$F(A, B) = \bar{A}B + AB = \Sigma(m_1, m_3) = \Sigma(1, 3)$$

A	B	Y	minterm	minterm name
0	0	0	$\bar{A} \bar{B}$	m_0
0	1	1	$\bar{A} B$	m_1
1	0	0	$A \bar{B}$	m_2
1	1	1	$A B$	m_3

- Example:

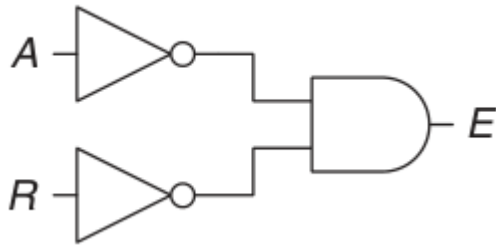
$$F(A, B) = (A + B)(\bar{A} + B) = \Pi(M_0, M_2) = \Pi(0, 2)$$

A	B	Y	maxterm	maxterm name
0	0	0	$A + B$	M_0
0	1	1	$A + \bar{B}$	M_1
1	0	0	$\bar{A} + B$	M_2
1	1	1	$\bar{A} + \bar{B}$	M_3

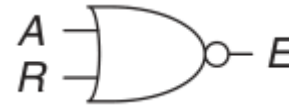
Boolean Algebra

Sum-of-Products form (SOP) and Product-of-Sums form (POS)

- A function can be represented by product of minterms or sum of maxterms
- Example:



$$E = \Pi(1,2,3)$$



$$E = \Sigma(0)$$

- SOP produces a shorter equation when the output is TRUE on only a few rows of the truth table.
- POS produces a shorter equation when the output is FALSE on only a few rows of truth table.

Boolean Algebra

Sum-of-Products form (SOP) and Product-of-Sums form (POS)

1. Minterm to Maxterm conversion:

rewrite minterm shorthand using maxterm shorthand
replace minterm indices with the indices not already used

$$\text{E.g., } F(A, B, C) = \sum m(3, 4, 5, 6, 7) = \prod M(0, 1, 2)$$

2. Maxterm to Minterm conversion:

rewrite maxterm shorthand using minterm shorthand
replace maxterm indices with the indices not already used

$$\text{E.g., } F(A, B, C) = \prod M(0, 1, 2) = \sum m(3, 4, 5, 6, 7)$$

3. Expansion of F to expansion of $\bar{F}$

$$\begin{array}{ll} \text{E.g., } F(A, B, C) = \sum m(3, 4, 5, 6, 7) & \longrightarrow \bar{F}(A, B, C) = \sum m(0, 1, 2) \\ = \prod M(0, 1, 2) & \longrightarrow = \prod M(3, 4, 5, 6, 7) \end{array}$$

4. Minterm expansion of F to Maxterm expansion of $\bar{F}$

rewrite in Maxterm form, using the same indices as F

$$\begin{array}{ll} \text{E.g., } F(A, B, C) = \sum m(3, 4, 5, 6, 7) & \longrightarrow \bar{F}(A, B, C) = \prod M(3, 4, 5, 6, 7) \\ = \prod M(0, 1, 2) & \longrightarrow = \sum m(0, 1, 2) \end{array}$$

Boolean Algebra

Axioms of Boolean Algebra

Axiom		Dual		Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binary field
A2	$\bar{0} = 1$	A2'	$\bar{1} = 0$	NOT
A3	$0 \bullet 0 = 0$	A3'	$1 + 1 = 1$	AND/OR
A4	$1 \bullet 1 = 1$	A4'	$0 + 0 = 0$	AND/OR
A5	$0 \bullet 1 = 1 \bullet 0 = 0$	A5'	$1 + 0 = 0 + 1 = 1$	AND/OR

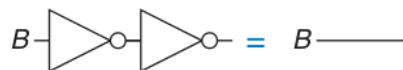
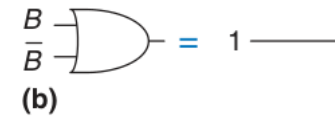
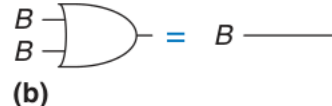
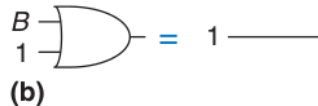
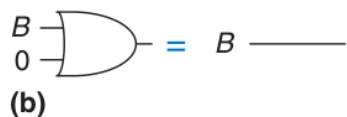
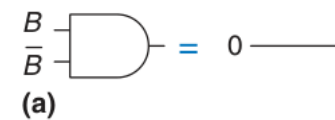
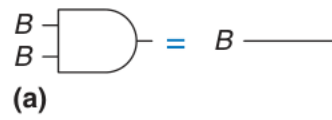
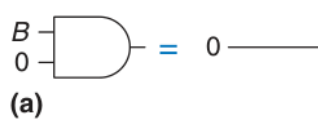
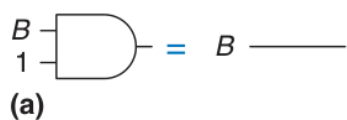
Axioms: Tiên đề.

Dual: Tính đối ngẫu

Boolean Algebra

One variable's theorems

	Theorem		Dual	Name
T1	$B \bullet 1 = B$	T1'	$B + 0 = B$	Identity
T2	$B \bullet 0 = 0$	T2'	$B + 1 = 1$	Null Element
T3	$B \bullet B = B$	T3'	$B + B = B$	Idempotency
T4		$\overline{\overline{B}} = B$		Involution
T5	$B \bullet \overline{B} = 0$	T5'	$B + \overline{B} = 1$	Complements



Boolean Algebra

Several variables' theorems

Theorem		Dual		Name
T6	$B \bullet C = C \bullet B$	T6'	$B + C = C + B$	Commutativity
T7	$(B \bullet C) \bullet D = B \bullet (C \bullet D)$	T7'	$(B + C) + D = B + (C + D)$	Associativity
T8	$(B \bullet C) + (B \bullet D) = B \bullet (C + D)$	T8'	$(B + C) \bullet (B + D) = B + (C \bullet D)$	Distributivity
T9	$B \bullet (B + C) = B$	T9'	$B + (B \bullet C) = B$	Covering
T10	$(B \bullet C) + (B \bullet \overline{C}) = B$	T10'	$(B + C) \bullet (B + \overline{C}) = B$	Combining
T11	$(B \bullet C) + (\overline{B} \bullet D) + (C \bullet D)$ $= (B \bullet C) + (\overline{B} \bullet D)$	T11'	$(B + C) \bullet (\overline{B} + D) \bullet (C + D)$ $= (B + C) \bullet (\overline{B} + D)$	Consensus
T12	$\overline{B_0 \bullet B_1 \bullet B_2 \dots}$ $= (\overline{B_0} + \overline{B_1} + \overline{B_2} \dots)$	T12'	$\overline{B_0 + B_1 + B_2 \dots}$ $= (\overline{B_0} \bullet \overline{B_1} \bullet \overline{B_2} \dots)$	De Morgan's Theorem

Boolean Algebra

DeMorgan's Law

$$\overline{(X + Y + Z + \dots)} = \bar{X} \cdot \bar{Y} \cdot \bar{Z} \cdot \dots$$

$$\overline{(X \cdot Y \cdot Z \cdot \dots)} = \bar{X} + \bar{Y} + \bar{Z} + \dots$$

$$A = \overline{(X + Y)} = \bar{X}\bar{Y}$$



X	Y	$\overline{X + Y}$	$\bar{X}$	$\bar{Y}$	$\bar{X}\bar{Y}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

NOR is equivalent to AND
with inputs complemented



$$B = \overline{(XY)} = \bar{X} + \bar{Y}$$



X	Y	$\overline{XY}$	$\bar{X}$	$\bar{Y}$	$\bar{X} + \bar{Y}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

NAND is equivalent to OR
with inputs complemented

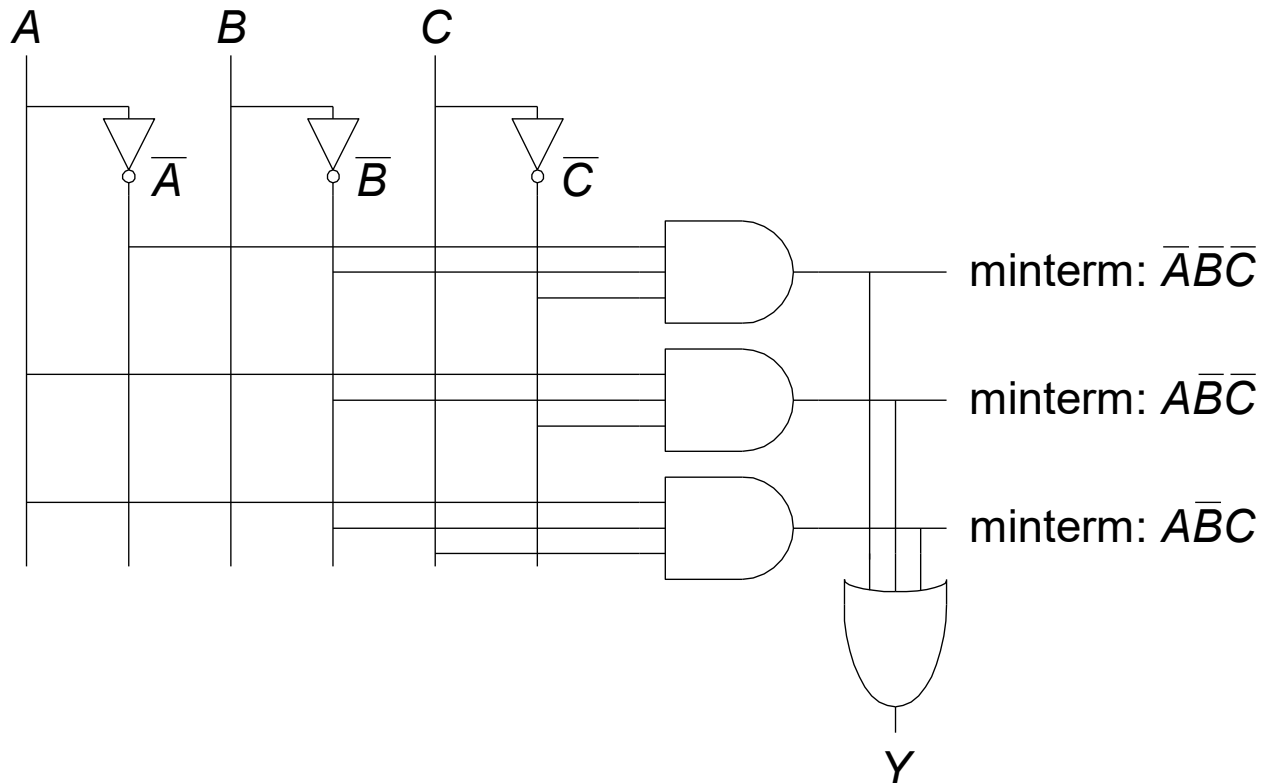


Represent a Logic Circuit

Logic and Gates

- Example:

$$Y = (\bar{A} \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot C)$$



Question: $Y = \Sigma(?)$

Determine the truth table of the function?

Represent a Logic Circuit

Notation

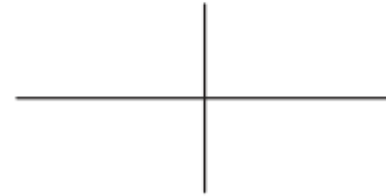
wires connect
at a T junction



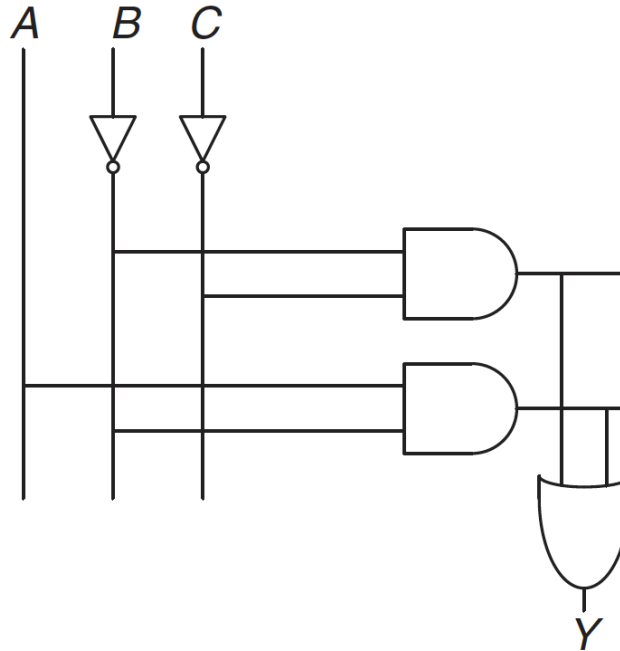
wires connect
at a dot



wires crossing
without a dot do
not connect



- Example:



Represent a Logic Circuit

Building Blocks

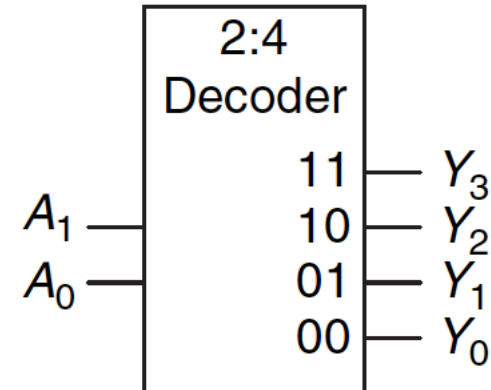
- Combinational logic is often grouped into larger building blocks to build more **complex systems**
- Hides the **unnecessary gate-level details** to emphasize the function of the building block
- We now examine:
 - Decoder
 - Multiplexer
 - Full adder
 - PLA (Programmable Logic Array)

Represent a Logic Circuit

Decoder

- “Input pattern detector”
- n inputs and 2^n outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The **output** that is logically 1 is the output corresponding to the input **pattern** that the logic circuit is expected to detect
- Example: 2-to-4 decoder

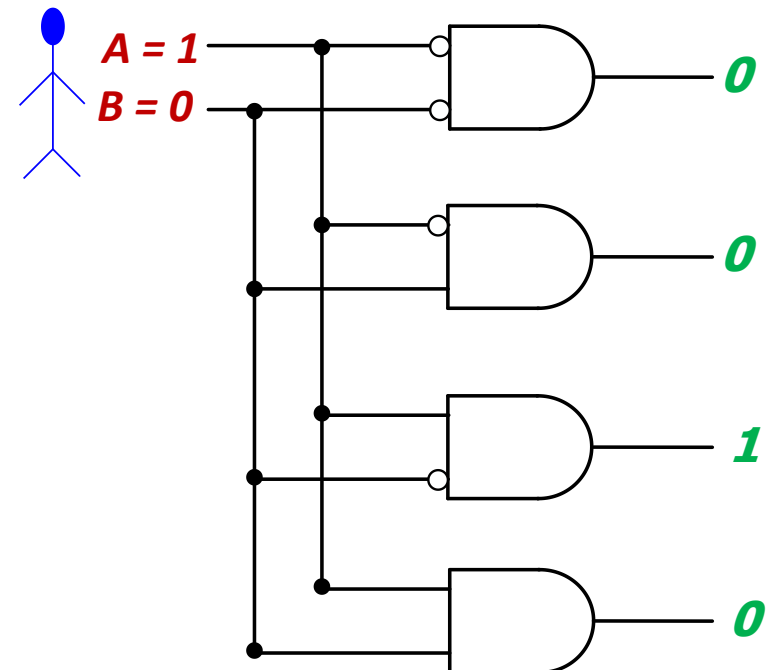
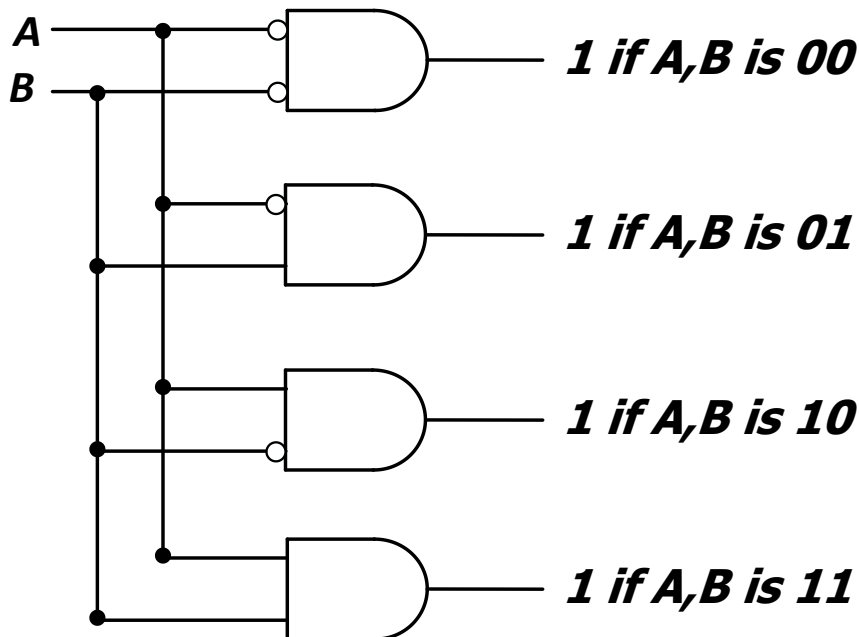
A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



Represent a Logic Circuit

Decoder

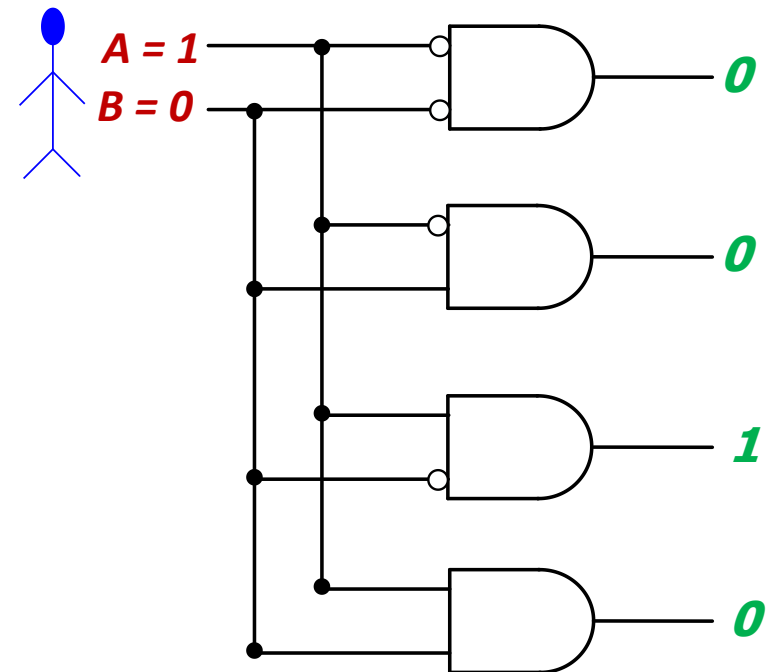
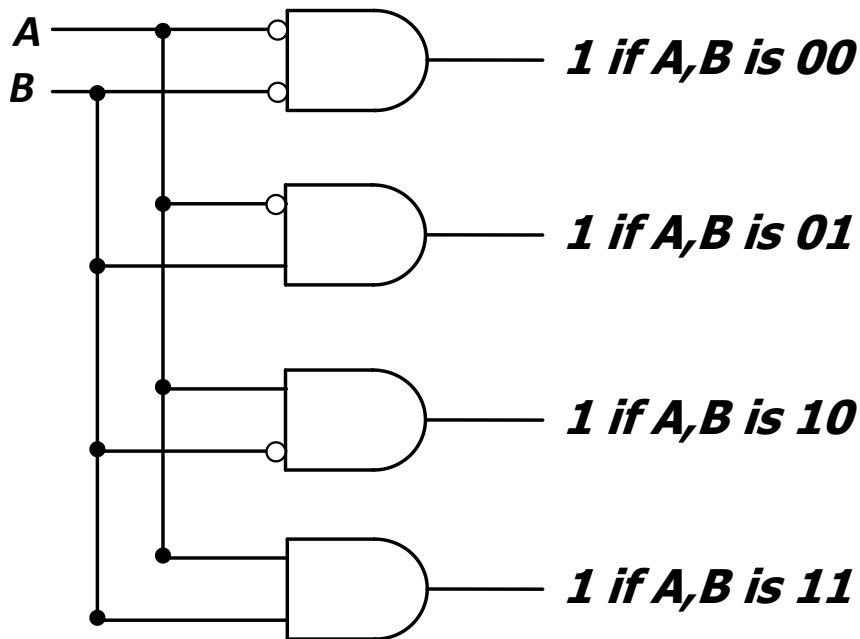
- n inputs and 2^n outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The **output** that is logically 1 is the output corresponding to the input **pattern** that the logic circuit is expected to detect



Represent a Logic Circuit

Decoder

- n inputs and 2^n outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The **output** that is logically 1 is the output corresponding to the input **pattern** that the logic circuit is expected to detect



Represent a Logic Circuit

Decoder

- Logic using decoder

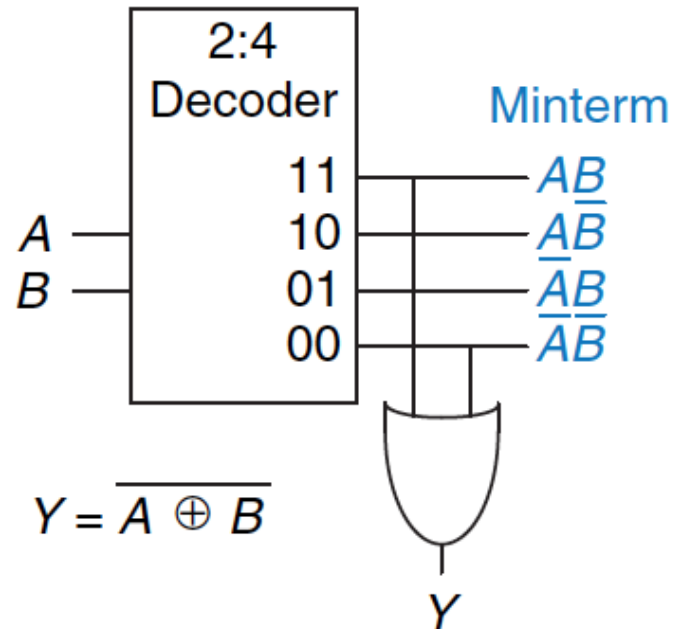


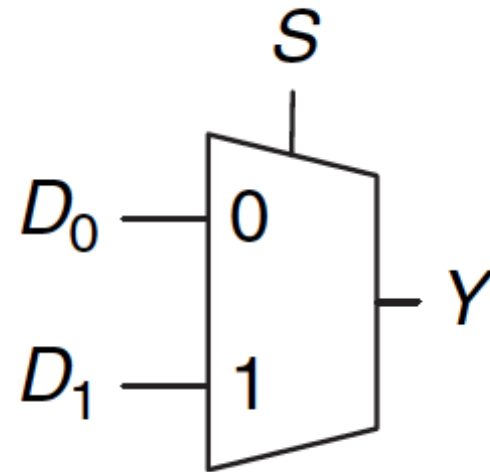
Figure 2.65 Logic function using decoder

Represent a Logic Circuit

Multiplexer (MUX), or Selector

- **Selects** one of the N inputs to connect it to the output
 - based on the value of a $\log_2 N$ -bit control input called **select**
- Example: 2-to-1 MUX

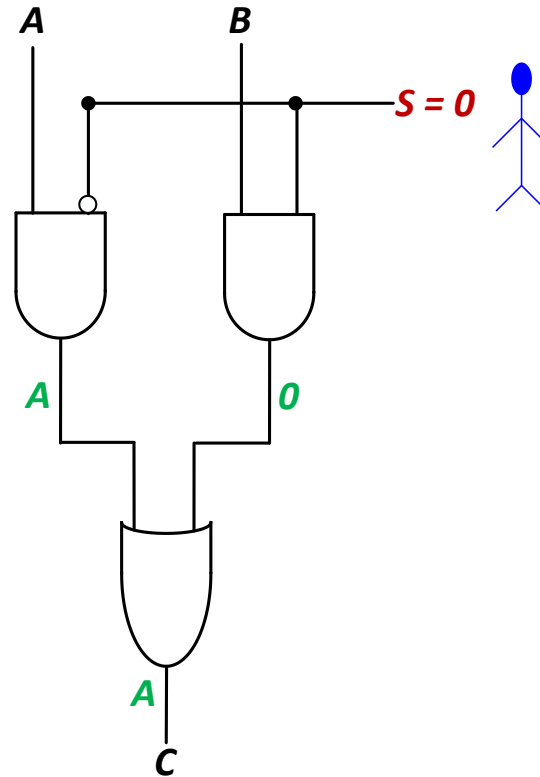
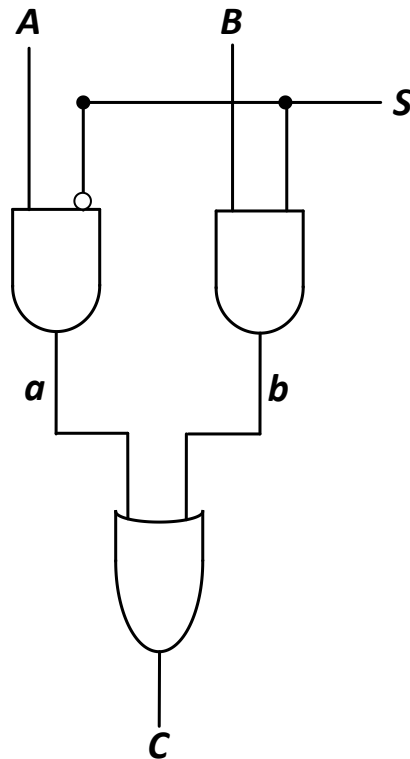
S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Represent a Logic Circuit

Multiplexer (MUX), or Selector

- **Selects** one of the N inputs to connect it to the output
 - based on the value of a $\log_2 N$ -bit control input called **select**
- Example: 2-to-1 MUX

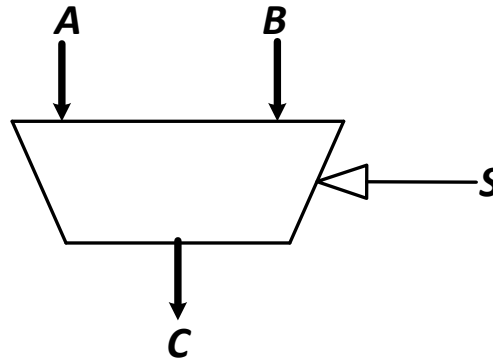


Represent a Logic Circuit

Multiplexer (MUX), or Selector

- The output C is always connected to either the input A or the input B
 - Output value depends on the value of the **select line S**

S	C
0	A
1	B

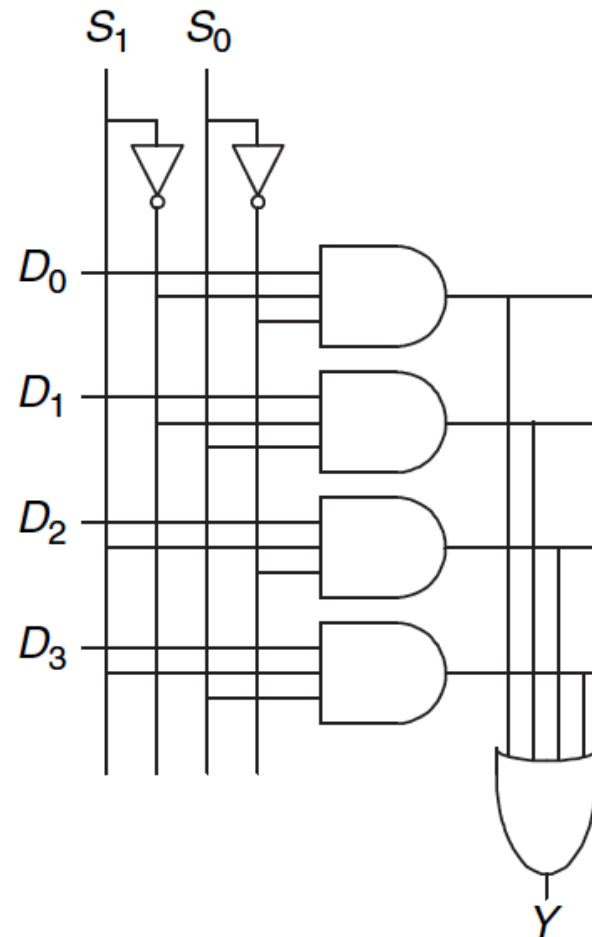
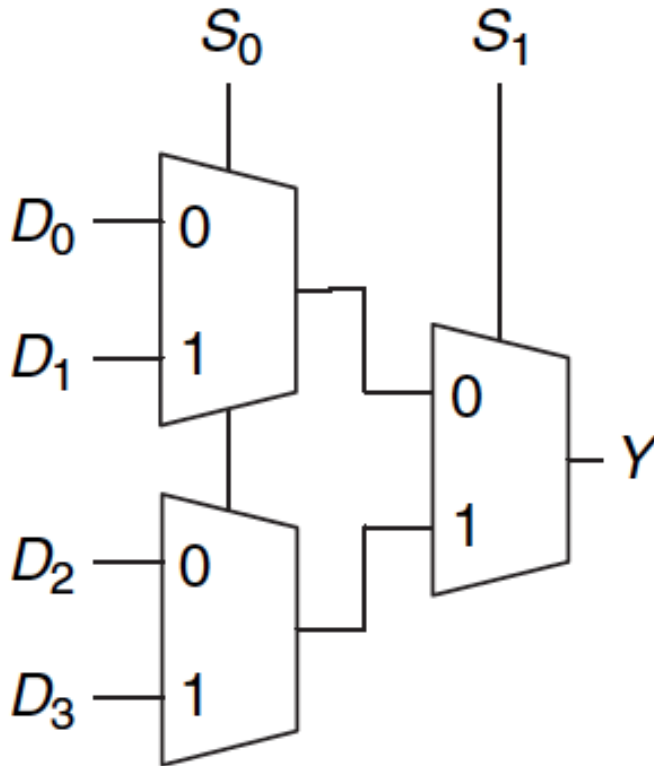


- **Your task:** Draw the schematic for a 4-input (4:1) MUX
 - Gate level: as a combination of basic AND, OR, NOT gates
 - Module level: As a combination of 2-input (2:1) MUXes

Represent a Logic Circuit

Multiplexer (MUX), or Selector

- Example: A 4-to-1 Multiplexer



...

Represent a Logic Circuit

Multiplexer (MUX), or Selector

- Multiplexers can be used as lookup tables to perform logic functions

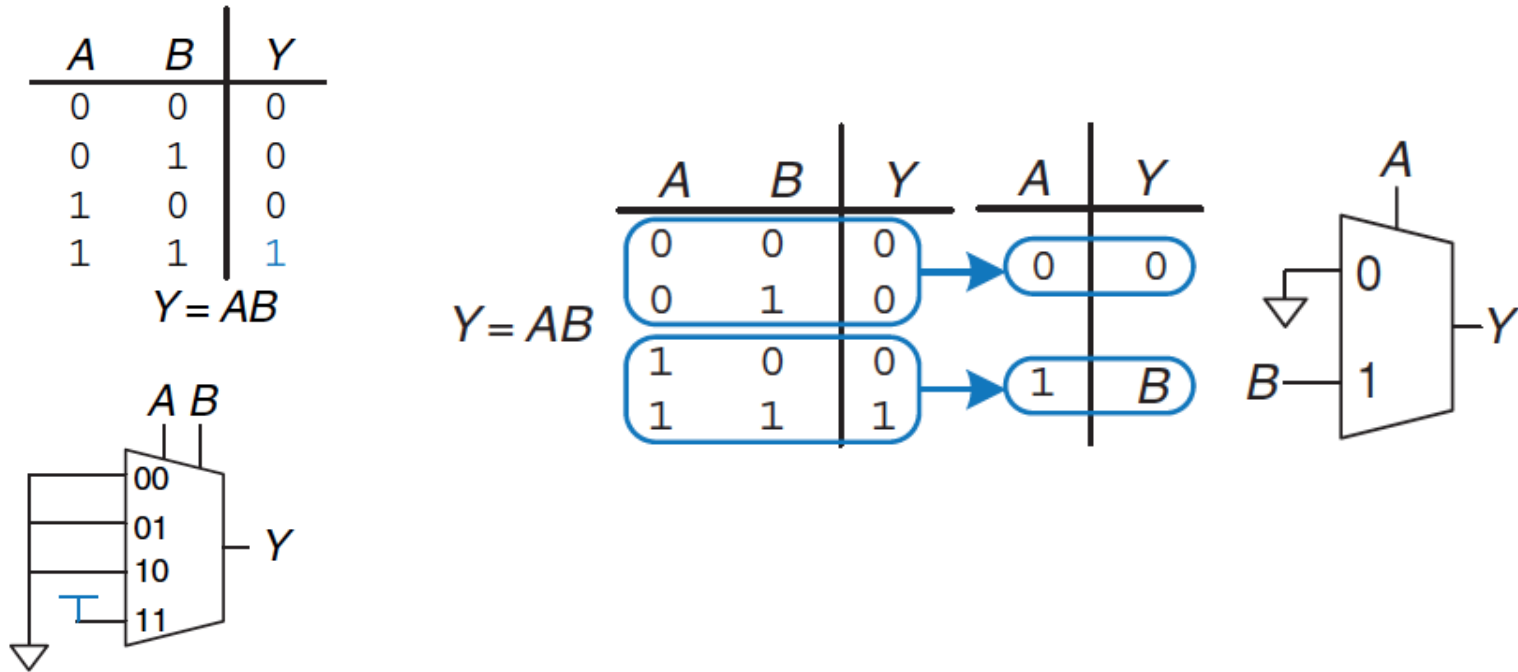


Figure 2.59 4:1 multiplexer implementation of two-input AND function

Represent a Logic Circuit

Multiplexer (MUX), or Selector

- Multiplexers can be used as lookup tables to perform logic functions

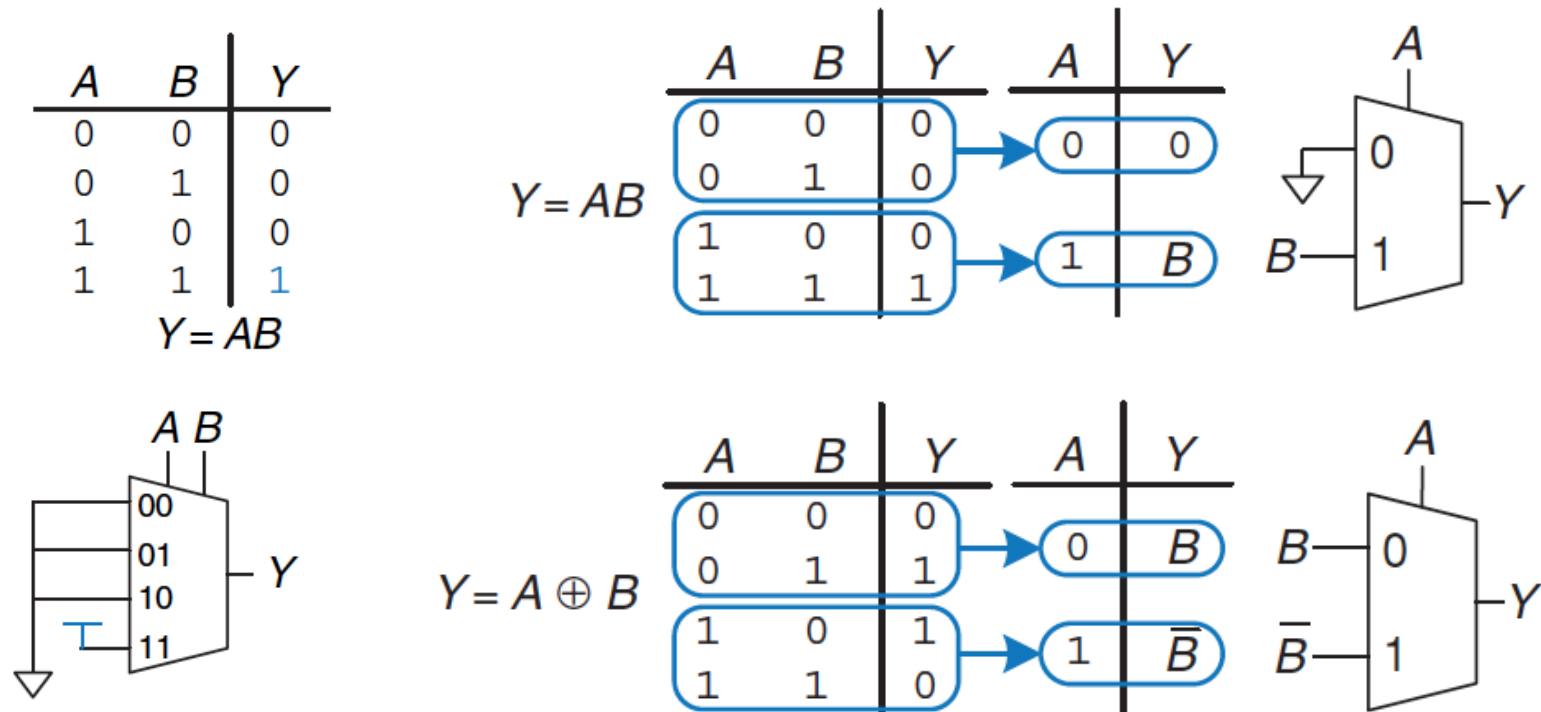


Figure 2.59 4:1 multiplexer implementation of two-input AND function

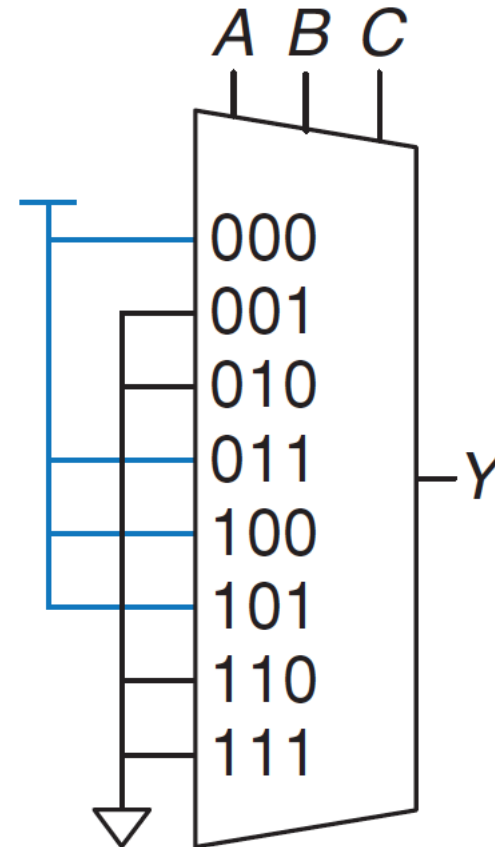
Represent a Logic Circuit

Multiplexer (MUX), or Selector

- Multiplexers can be used as lookup tables to perform logic functions

<i>A</i>	<i>B</i>	<i>C</i>	<i>Y</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = A\bar{B} + \bar{B}\bar{C} + \bar{A}BC$$

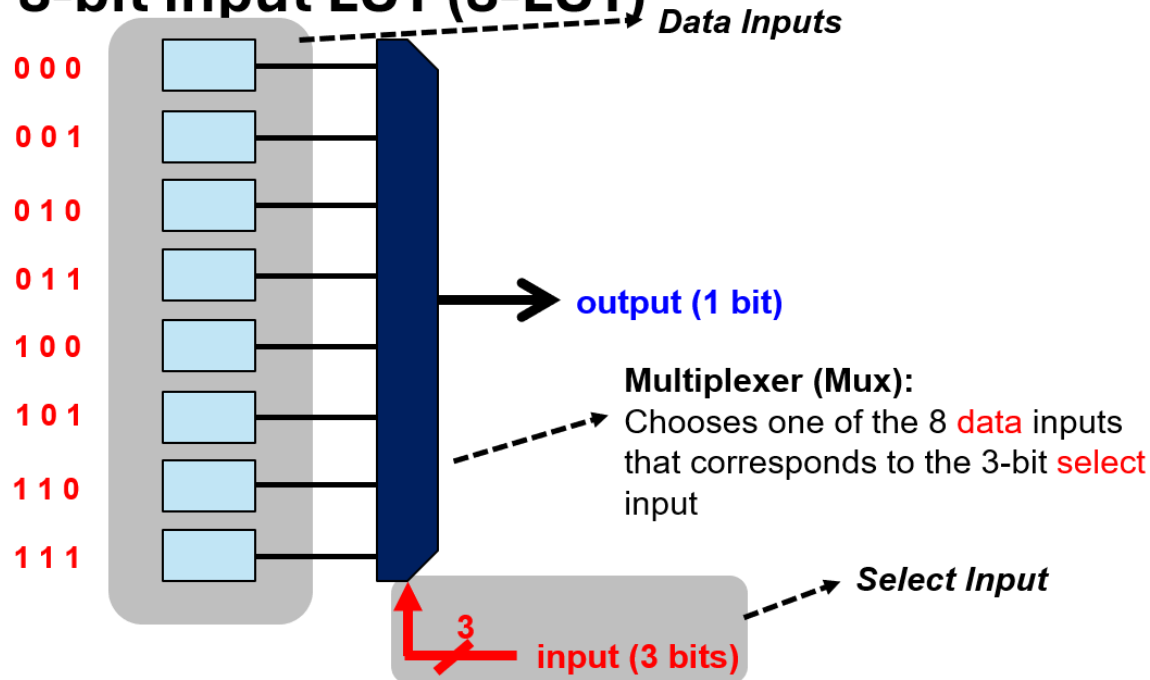


Represent a Logic Circuit

Multiplexer (MUX), or Selector

- Multiplexers can be used as lookup tables to perform logic functions

- 3-bit input LUT (3-LUT)**



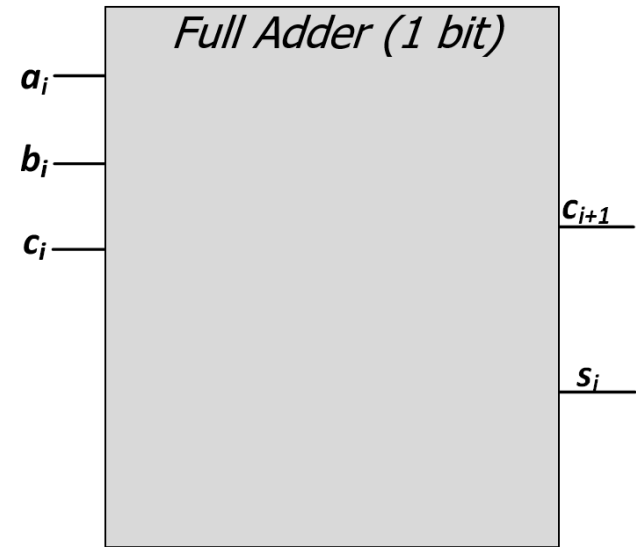
3-LUT can implement
any 3-bit input function

Represent a Logic Circuit

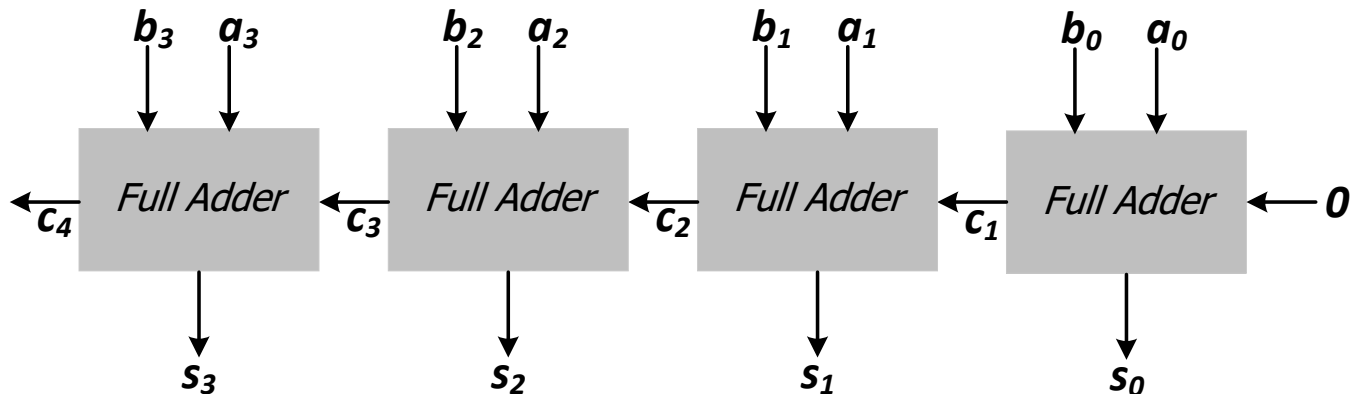
Full-Adder

1-bit addition

a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

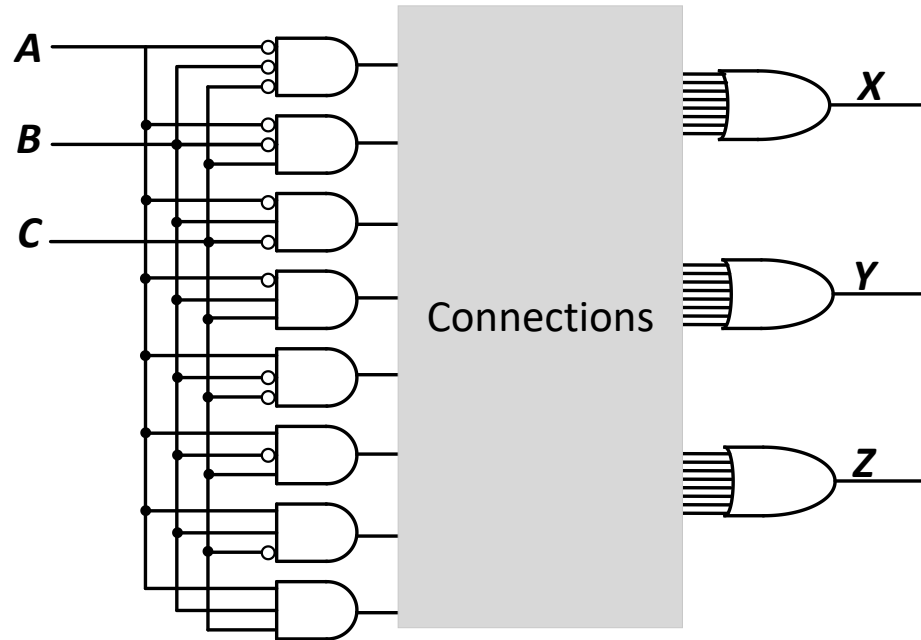


4-bit addition



Represent a Logic Circuit

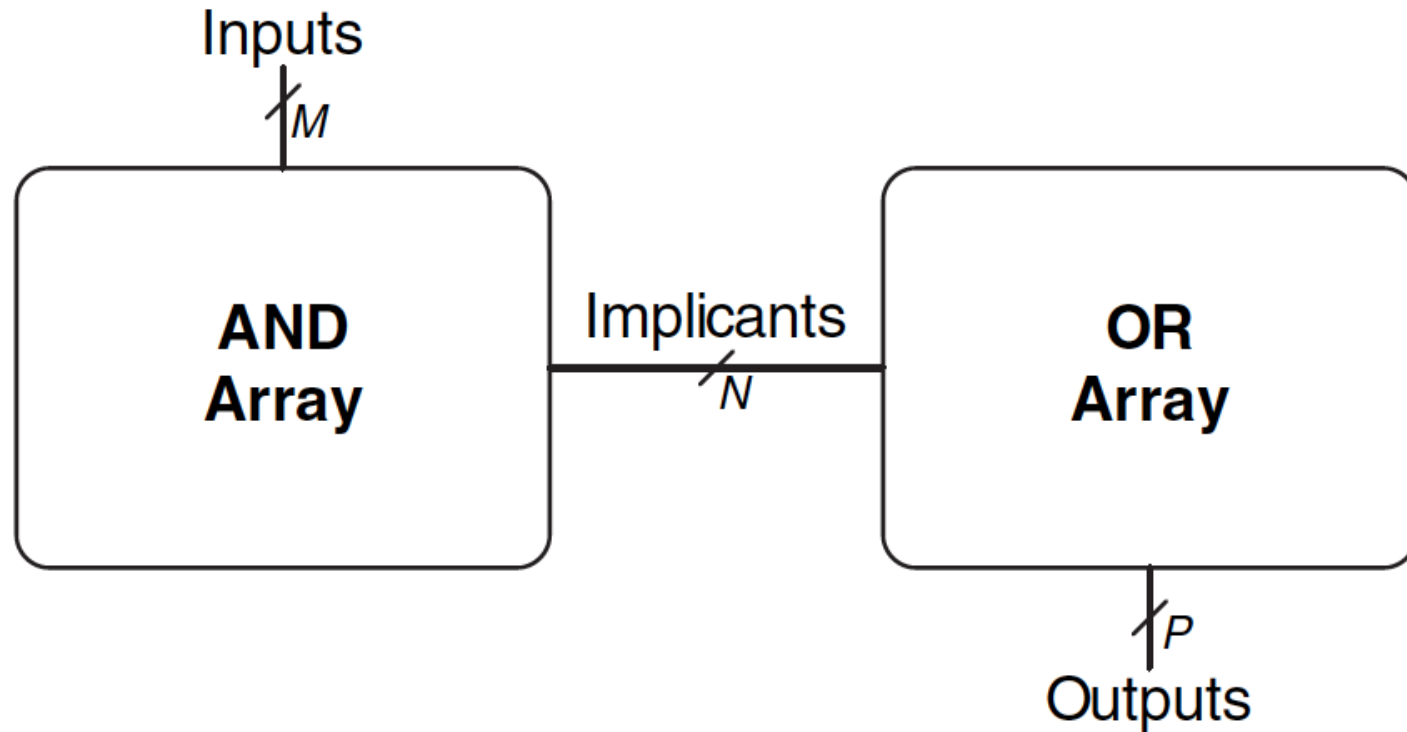
PLA



- ❑ An **array** of AND gates followed by an **array** of OR gates
- ❑ For an n -input logic function, we need a PLA with 2^n n -input AND gates
- ❑ The number of OR gates is determined by the number of output columns in the truth table

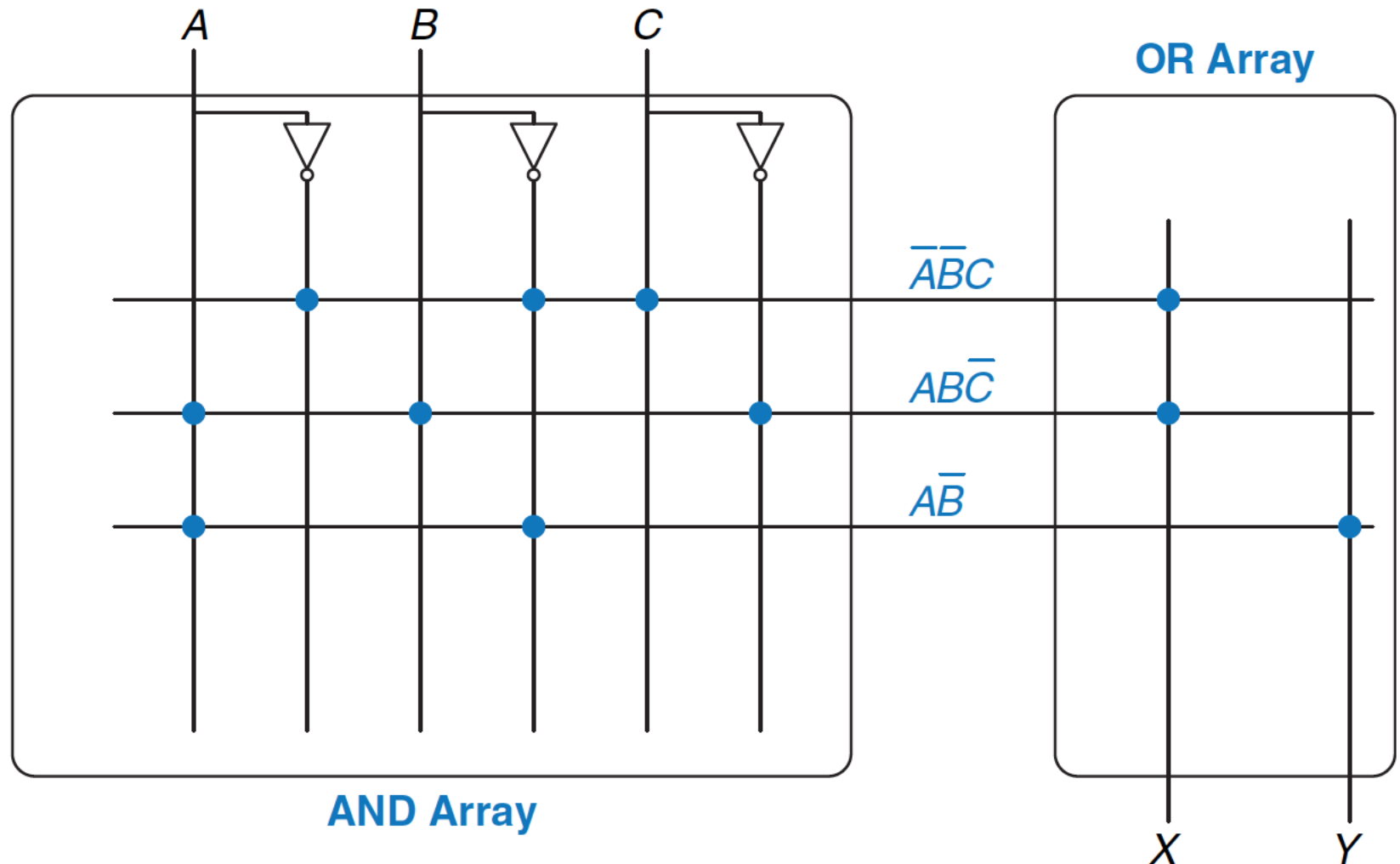
Represent a Logic Circuit

PLA



Represent a Logic Circuit

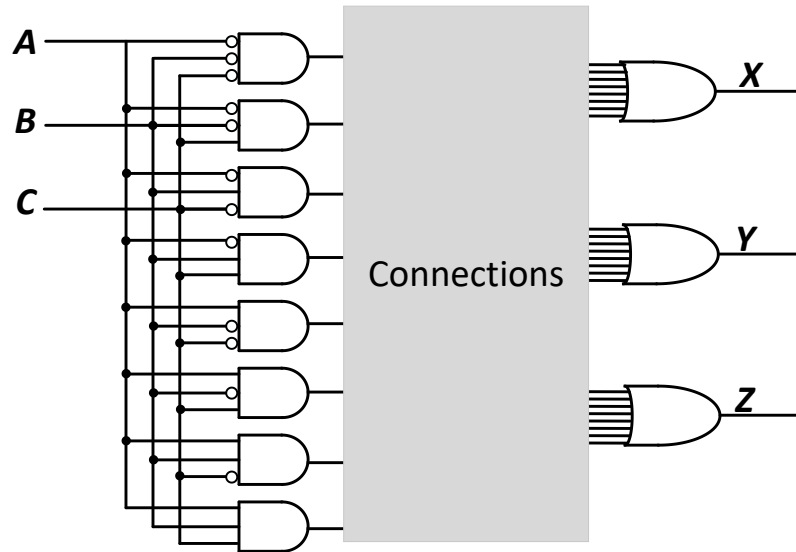
PLA



Represent a Logic Circuit

PLA

Implementing a Full Adder Using a PLA



a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

This input should not be connected to any outputs

We do not need this output

